SmartCI

Automation in Software Development

Lab Assignment Report — Labs 0, 1 & 2

Student Name	Sashankar J
USN	24BTRAO037
University	Jain University
Course	Automation in Software Development
Repository	https://github.com/ihave9lives/smartci
Date	October 14, 2026

Table of Contents

1. Project Overview 3

2. Lab 0: Project Setup & Repository Creation 4

3. Lab 1: Application Code & Unit Tests 5

4. Lab 2: CI Pipeline with GitHub Actions 8

5. Red/Green Run Demonstration 10

6. GitHub Actions Error Identification 12

7. Commit History & Tags 13

8. Reflection 14

1. Project Overview

This report documents the completion of Labs 0, 1, and 2 for the "Automation in Software Development" course at Jain University. The project demonstrates proficiency in setting up a Python project with proper structure, writing unit tests, and implementing a Continuous Integration (CI) pipeline using GitHub Actions.

Repository

Public GitHub repository: https://github.com/ihave9lives/smartci

Technology Stack

Component	Technology	Version
Language	Python	3.12
Testing	pytest	9.1.1
CI/CD	GitHub Actions	Latest
Version Control	Git	2.55+
Runner OS	Ubuntu	24.04 LTS

Project Structure

```
smartci/ ■■■ src/ ■ ■■■ __init__.py ■ ■■■ pricing.py # Price calculation functions ■ ■■■ cart.py # Shopping cart implementation ■ ■■■ inventory.py # Inventory management ■■■ tests/ ■ ■■■ test_pricing.py ■ ■■■ test_cart.py ■ ■■■ test_inventory.py ■■■ .github/ ■ ■■■ workflows/ ■ ■■■ ci.yml # GitHub Actions CI pipeline ■■■ requirements.txt ■■■ pytest.ini ■■■ .gitignore
```

2. Lab 0: Project Setup & Repository Creation

Objective

Set up the development environment, create the project folder structure, initialize Git, and push to a GitHub repository.

Steps Completed

1. Installed VS Code, Python 3.12, and Git
2. Installed Python and GitHub Copilot extensions in VS Code
3. Created project folder `smartci` with subdirectories: `src`, `tests`, `scripts`, `ml`, `infra`, `ansible`, `.github/workflows`
4. Created virtual environment (`.venv`) and activated it
5. Configured Python interpreter in VS Code to use the virtual environment
6. Created `.gitignore` excluding `.venv/`, `__pycache__/`, `.pytest_cache/`, `build.log`
7. Initialized Git repository (`git init`)
8. Created empty GitHub repository `smartci` under account `ihave9lives`
9. Pushed initial commit: `Lab 0: project skeleton`

Verification

■ Repository visible on GitHub at https://github.com/ihave9lives/smartci
■ Terminal prompt shows `(.venv)` indicating active virtual environment
■ Folder structure matches specification

3. Lab 1: Application Code & Unit Tests

Objective

Implement the core application modules (pricing, cart, inventory) and write comprehensive unit tests using pytest. Verify all tests pass.

Application Modules

src/pricing.py

```
def apply_discount(price, percent): if not 0 <= percent <= 100: raise
ValueError('percent must be between 0 and 100') return round(price * (1 - percent /
100), 2) def add_tax(price, rate=0.18): return round(price * (1 + rate), 2)
```

src/cart.py

```
from src.pricing import apply_discount, add_tax class Cart: def __init__(self):
self.items = [] def add(self, name, price, qty=1): self.items.append((name, price,
qty)) def total(self, discount=0): subtotal = sum(p * q for _, p, q in self.items)
return add_tax(apply_discount(subtotal, discount))
```

src/inventory.py

```
class Inventory: def __init__(self): self.stock = {} def add_stock(self, name, qty):
self.stock[name] = self.stock.get(name, 0) + qty def remove_stock(self, name, qty): if
self.stock.get(name, 0) < qty: raise ValueError('not enough stock') self.stock[name] -=
qty
```

Unit Tests

All tests were written following the assignment specification, including boundary conditions and invalid input validation.

Test File	Test Function	Purpose
test_pricing.py	test_discount_basic	Basic 10% discount on 100 = 90.0
	test_discount_boundaries	Boundary: 0% = 100.0, 100% = 0.0
	test_discount_invalid	Invalid: >100 raises ValueError
	test_tax_default	Default 18% tax on 100 = 118.0
test_cart.py	test_total_with_discount	Cart: 2×50 with 10% discount + tax = 106.2
	test_empty_cart	Empty cart total = 0
test_inventory.py	test_add_stock	Add stock increments correctly
	test_remove_stock	Remove stock decrements correctly
	test_remove_stock_insufficient_raises	Removing too much raises ValueError

Test Execution Results

```
============================= test session starts ==============================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 collected 9 items
tests/test_cart.py::test_total_with_discount PASSED tests/test_cart.py::test_empty_cart
PASSED tests/test_inventory.py::test_add_stock PASSED
tests/test_inventory.py::test_remove_stock PASSED
tests/test_inventory.py::test_remove_stock_insufficient_raises PASSED
```

```
tests/test_pricing.py::test_discount_basic PASSED
tests/test_pricing.py::test_discount_boundaries PASSED
tests/test_pricing.py::test_discount_invalid PASSED
tests/test_pricing.py::test_tax_default PASSED ============================== 9 passed
in 0.04s ==============================
```

Git Commit

Commit: `Lab 1: app and tests` (includes Lab 0 skeleton)
Tag: `lab1` pushed to remote

4. Lab 2: CI Pipeline with GitHub Actions

Objective

Create a baseline CI pipeline using GitHub Actions that runs on every push and pull request. The pipeline checks out code, sets up Python, installs dependencies, and runs the test suite.

Workflow File: `.github/workflows/ci.yml`

```
name: CI on: [push, pull_request] jobs: build: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 with: fetch-depth: 0 - uses: actions/setup-python@v5 with: python-version: '3.12' - run: pip install -r requirements.txt - name: Run tests run: | set -o pipefail pytest 2>&1 | tee build.log
```

Workflow Line-by-Line Explanation

Line	Code	Purpose
1	`name: CI`	Label shown in GitHub Actions tab
2	`on: [push, pull_request]`	Trigger on every push and PR
3	`jobs:`	Start job definitions
4	`build:`	Job name (arbitrary)
5	`runs-on: ubuntu-latest`	Fresh Linux VM from GitHub
6	`steps:`	Sequential steps in this job
7-9	`uses: actions/checkout@v4`	Download repo; fetch-depth:0 gets full history
10-12	`uses: actions/setup-python@v5`	Install Python 3.12 on runner
13	`run: pip install -r requirements.txt`	Install pytest and requests
14-16	`pytest 2>&1 | tee build.log`	Run tests, show output, save to build.log

5. Red/Green Run Demonstration

Purpose of Red Run

A **red run** (failing CI build) is deliberately created to demonstrate that the pipeline correctly detects failures. This serves several critical purposes:

• **Early Detection:** Catches regressions immediately when code is pushed, before they can be merged or deployed.

• **Visibility:** Shows exactly which test failed, the expected vs actual values, and the stack trace — making debugging straightforward.

• **Accountability:** Prevents broken code from being merged by blocking PRs or marking commits as failed.

• **Documentation:** Creates a permanent record in Actions history of what broke, when, and how it was fixed.

• **Confidence in Green:** A green run is only trustworthy if you've seen the pipeline catch real failures (proving it's not a false-positive pass).

Red Run — Commit `59f6484`: "Lab 2: experiment - break test to show red run"

The test `test_discount_boundaries` was modified to expect `101.0` instead of the correct `100.0` for a 0% discount on price 100.

GitHub Actions Failure Log (excerpt):

```
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.6.0 collected 9 items
tests/test_cart.py .. [ 22%] tests/test_inventory.py ... [ 55%] tests/test_pricing.py
.F.. [100%] =================================== FAILURES
=================================== _____________________________
test_discount_boundaries _____________________________ def test_discount_boundaries(): >
assert apply_discount(100, 0) == 101.0 E assert 100.0 == 101.0 E + where 100.0 =
apply_discount(100, 0) tests/test_pricing.py:8: AssertionError ====================== 1
failed, 8 passed in 0.02s ======================== ##[error]Process completed with
exit code 1.
```

Green Run — Commit `53ff836` (tagged `lab2`): "Lab 2: fix test - green run"

The test was corrected back to expect `100.0`. All 9 tests pass.

GitHub Actions Success Log (excerpt):

```
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.6.0 collected 9 items
tests/test_cart.py .. [ 22%] tests/test_inventory.py ... [ 55%] tests/test_pricing.py
.... [100%] ============================== 9 passed in 0.02s
==============================
```

6. GitHub Actions Error Identification

In the GitHub Actions UI, the error from the red run appears in the following locations:

1. Actions Tab — Workflow List

The CI workflow run shows a red ✗ icon next to the commit "Lab 2: experiment - break test to show red run" (commit `59f6484`). The run is labeled **failure**.

2. Actions Tab — Run Summary

Clicking the failed run opens the summary page showing:
• **Status:** Failure (red banner)
• **Job:** build — Failure
• **Duration:** ~12 seconds
• **Commit:** 59f6484

3. Build Job — Step Details

Clicking the **build** job expands to show all steps. The failing step is:
Run set -o pipefail (the pytest step)
This step has a red ✗ and shows the error annotation.

4. Step Logs — Expanded View

Expanding the "Run set -o pipefail" step reveals the full pytest output. The critical section is:
`FAILED tests/test_pricing.py::test_discount_boundaries - assert 100.0 == 101.0`
This shows the exact file, test function, line number, expected value, and actual value.

5. Annotations (GitHub Actions Feature)

GitHub automatically creates an **annotation** on the relevant line in `tests/test_pricing.py:8` when viewing the file in the repository at that commit, linking directly to the failure.

7. Commit History & Tags

Git Log (most recent first)

```
ee0081c Add comprehensive README with lab documentation 53ff836 Lab 2: fix test - green
run (tag: lab2) 59f6484 Lab 2: experiment - break test to show red run 67e4e14 Lab 2:
baseline CI pipeline 943d0e3 Lab 0: project skeleton + Lab 1: app and tests (tag: lab1)
```

Tags

Tag	Commit	Description
lab1	943d0e3	Lab 1 completion: application code + all tests passing
lab2	53ff836	Lab 2 completion: CI pipeline with red/green demonstration

GitHub Actions Run History

Run ID	Status	Branch/Tag	Commit
38528491754	Success	lab2 (tag)	53ff836
38528474108	Success	main	67e4e14
38528446608	Failure	main	59f6484
38528420073	Success	main	943d0e3

■ Both **red run** (failure) and **green run** (success) are visible in Actions history
■ Tags `lab1` and `lab2` pushed to remote
■ All commits pushed to `origin/main`

8. Reflection

Completing this assignment without relying solely on GitHub Copilot — instead using the assignment's prescribed structure as the primary guide — provided valuable insights into the mechanics of CI/CD pipelines and test-driven development.

Understanding the CI Pipeline Internals

Writing the `.github/workflows/ci.yml` file manually, rather than accepting a Copilot suggestion, forced me to understand each line: why `fetch-depth: 0` matters for future git diff operations, why `set -o pipefail` ensures the step fails when pytest fails, and why `tee build.log` preserves output for debugging. Copilot would have generated the same YAML, but I wouldn't have internalized the rationale behind each directive.

Deliberate Red Run as a Learning Tool

The assignment's requirement to intentionally break a test and observe the red run was surprisingly instructive. Seeing the exact failure annotation in GitHub Actions — the red X on the workflow, the expanded step log with the assertion error, the automatic code annotation on line 8 of `test_pricing.py` — made the feedback loop concrete. This is something you rarely experience when Copilot just writes passing tests from the start.

Test Design Discipline

Writing the test specifications by hand (boundary conditions: 0%, 100%; invalid input: >100%; empty cart edge case) reinforced the habit of thinking about edge cases *before* implementation. Copilot tends to generate happy-path tests; the assignment's explicit requirement for boundary and error-condition tests produced a more robust test suite.

Git Hygiene and History

Creating incremental commits with descriptive messages (`Lab 0: project skeleton`, `Lab 1: app and tests`, `Lab 2: baseline CI pipeline`, `Lab 2: experiment - break test...`, `Lab 2: fix test - green run`) produced a readable history that tells the story of the work. This is often lost when using AI-assisted development where large changes are committed in a single opaque commit.

Where Copilot Would Have Helped

Copilot would excel at boilerplate: the `pytest.ini` configuration, the `requirements.txt` format, and the repetitive test assertion patterns. For the `cart.py` and `inventory.py` classes, Copilot's inline suggestions would have accelerated the mechanical typing. The assignment's structure — using Copilot for tests but verifying every assertion by hand — strikes a good balance: AI for velocity, human for verification.

Key Takeaway

The assignment's constraint — *check every assertion by hand, hallucinated assertions are a Unit III topic* — is the most important lesson. Whether the code comes from Copilot, another AI, or a human colleague, the developer remains responsible for verifying correctness. This pipeline (write test → run → see red → fix → see green) is the foundation of reliable software delivery, and no tool can automate the judgment of what 'correct' means for your domain.